[RFR 2027 Electronics] New Member Project Design Report

Team 1: Hank Habashy (hnh34), Rebekah Han (reh129), Shiven Jariwala (sjj105), and Dhruv Ajwani (daa307)

1. Revision Log

Editor	Date	Revision
Rebekah Han	2026-10-16	1

2. Purpose

To build a temperature sensor, taking analog input and then converting it into digital to output them over the I2C bus.

3. Parties Involved

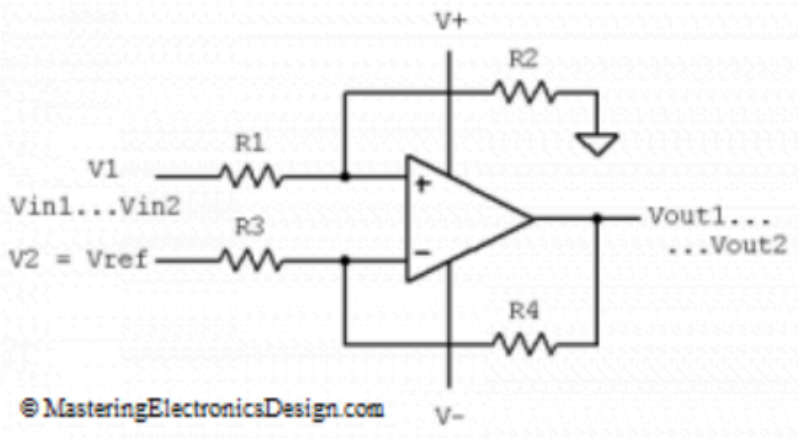
Name	Team Role	Report Role
Rebekah Han	New Member	Author
Hank Habashy	New Member	
Shiven Jariwala	New Member	
Dhruv Ajwani	New Member	

4. Research

Firstly, the temperature diodes involved following a similar setup as to what was written in the datasheet. But since we are using a 5V source and aimed to achieve a forward (typical) current of 1 mA, our needed resistor ended up being 3.1 kΩ. Given that I_out = [V_cc - V_out] / R, and V_cc = 5, V_out = 1.86, and I_out = 0.001 A, R = 3.1 kΩ.

Given that our temperature sensors' output voltage ranges from 1.30 V to 2.44 V and that the Arduino's ADC has 10-bit resolution with a default reference voltage of 5 V, we can shift and amplify the signals from the temperature sensors in such a way that its range becomes 1 - V. Keeping this in mind, we can also use an RC low-pass filter to keep lower frequency ranges since temperature signals change slowly in comparison to other signals. This will help filter out any electrical noise. Using this low pass filter calculator, to achieve a cutoff frequency at ~10 kHz, our resistor would need to be 1.6 kΩ and the capacitance would need to be 0.01 µF.

To handle amplification, a differential op-amp can be used to handle both level shifting and amplification. We would like the lowest temperature (i.e. -40°C) to be 4 V instead of the original 2.44 V (from the diode) and the highest temperature (120°C) to be 1 V instead of 1.30 V. Thus the gain for the op-amp will be 2.63 (= [4V - 1V]/[2.44V - 1.3V]). We used this differential amplifier calculator to help us build the op-amp circuit. The image below shows the circuit and the values to use.



Given Input Range

Vin1 = 1.3 V Vin2 = 2.44 V

Required Output Range

Vout1 = 1 V Vout2 = 4 V

Choose a Reference Voltage

V2 = 1.48 V

Choose R2 and R3

R2 = 10.00 kOhm R3 = 10.00 kOhm

Calculate R1 and R4

R1 = 0.02 kOhm R4 = 16.36 kOhm

In addition, to handle obtaining the V_REF = 1.48 V, we plan to use a voltage divider. The V_REF was computed by using the given equation: V_out = A(V_in - V_ref) + V_ref, where A is the gain. In our case, V_in = 1.3 V and V_out = 1.0 V.

The multiplexer will handle from what diode we will read from at a time. The 16:1 multiplexer has 4 select pins, which allow for addressing in a way. It is helpful to think of the 4 pins corresponding to a number in binary (e.g. 0000 = 0, 0001 = 1, 0010 = 2, etc). Each number corresponds to one input pin. So if the select pins were set to 0000, then the output of the multiplexer would return data from I_0 or input "pin" 0. We can take advantage of this addressing with the Arduino Nano. From the Nano, we can send 1s or 0s to the 4 select pins of the multiplexer. A program can be written to cycle through all of the temperature diodes from 0 (0000) through 15 (1111) and grab the output every time a new set of bits is sent to the multiplexer. From there calculating the highest temperature, average temperature, and other analysis can be done.

The 12-to-5 V step-down converter is able to put out 2.5 W. By summing the power consumption of elements connected to the 5V line, the estimated sum turns out to be 1.78 W, which is < 2.5 W. Since each temperature diode is 3.1 kΩ and is receiving 5 V, all 16 diodes dissipate about 0.129 W. The multiplexer consumes a max of 0.00004 W. The voltage divider consumes 0.00735 W. The op-amp is estimated to consume 0.05 W when the V_s is 15 V. Since ours is at 5 V, we will consume less power than 0.05 W. The Arduino Nano is estimated to consume a total of 1.6 W in our case because we are using 5 digital pins. Someone calculated that the Nano consumes 1.4 W to turn on, as the datasheet states the typical power consumption is TBC. The additional 0.2 W comes from 5 digital pins consuming each 40 mA max. Thus, summing all of the numbers comes out to 1.78 W.

5. Findings

Parts

Given Parts

- 16:1 multiplexer CD74HC4067
- Arduino Nano
- 12 to 5 V DCDC
- 16 temperature-sensitive diodes
- Red LED
- Breadboard
- Jumper cables
- Resistors

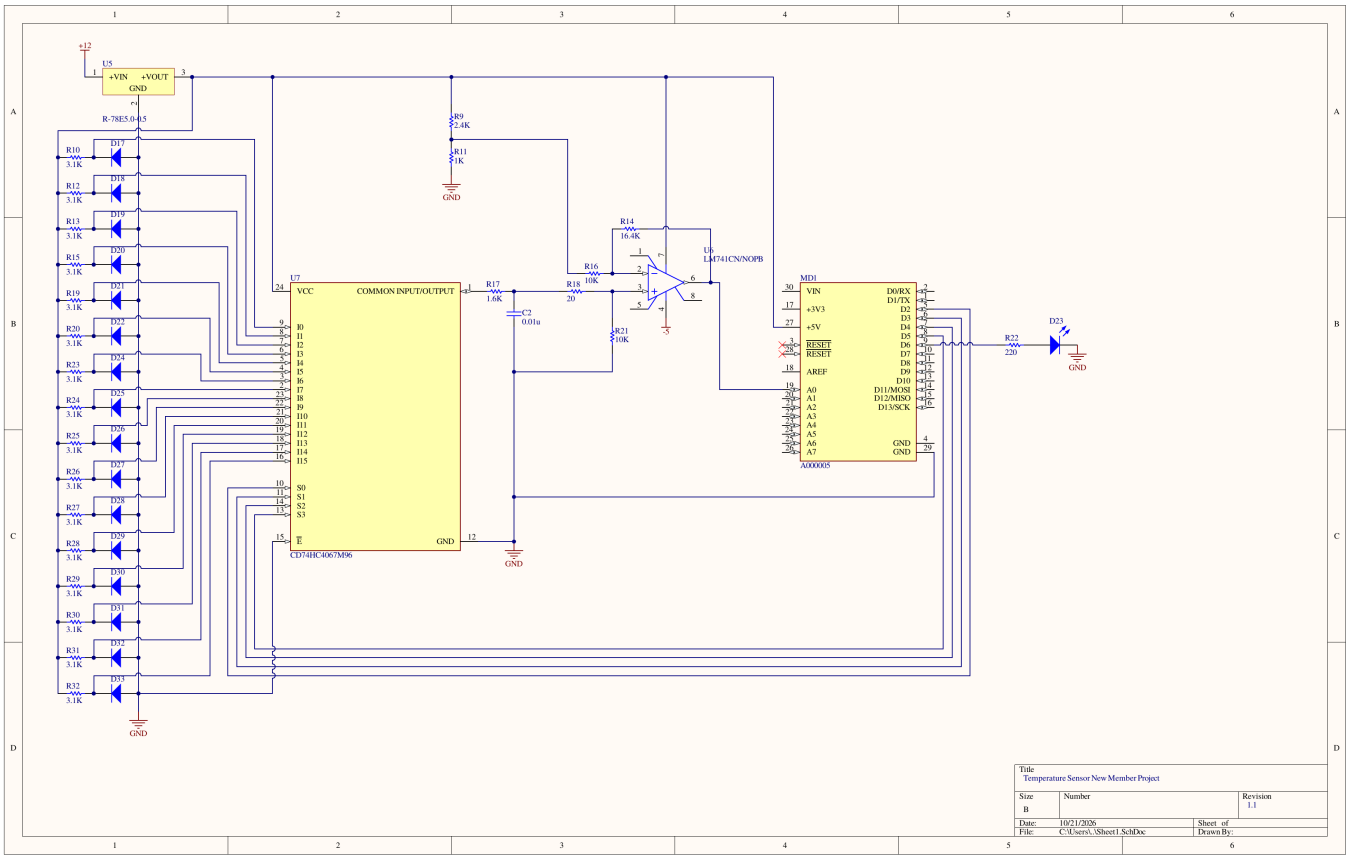
Additional Parts

The following lists parts not listed in the writeup, but are a part of our design:

- Op-amp LM741
- Capacitor: 0.01 µF
- 12V to 5V converter: Recom R-78E5.0-0.5

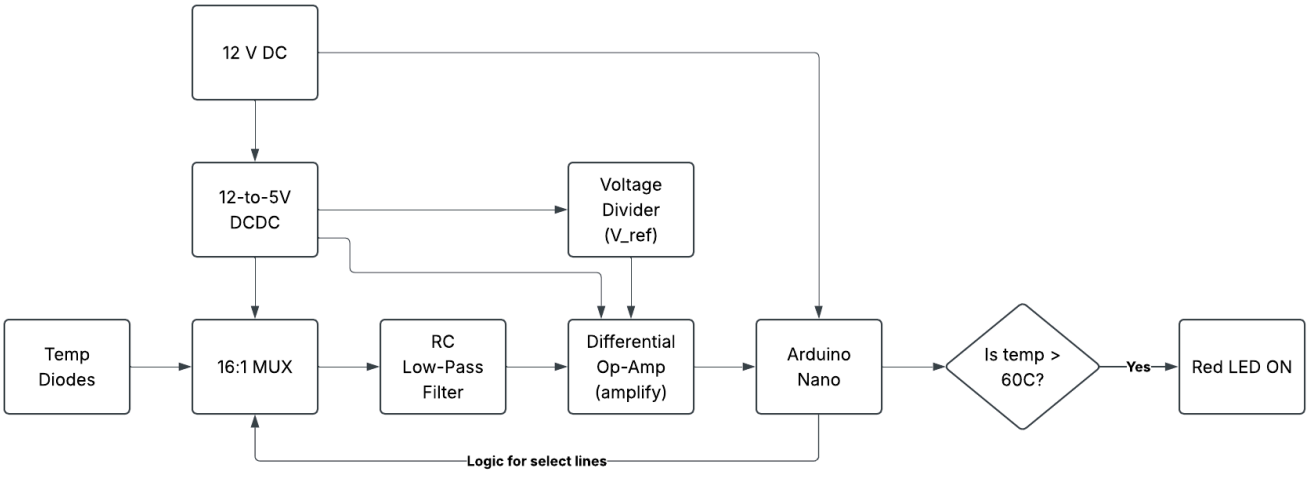
Schematic

The image below features our schematic.



Schematic Flow Chart

The image below is our schematic flow chart. The analog signal from the temperature diodes are fed through the multiplexer, which gets through the RC low pass filter, is then amplified, and then fed to the Arduino Nano. The signal is translated into a digital signal, which will be made available over I2C on a digital pin. If the temperature is > 60°C, then the red LED will turn on.



Code

The following is what we have so far for the code. Everything but linear interpolation has been implemented. Bit shifting is used to create 0 through 15 in binary for the select lines.

```
#include <Wire.h>

const int TEMP_PIN = 0;
const int LED_PIN = 13;
const int NUM_TEMP_SENSORS = 16;

// Temperature when the ADC is at 0V in Celsius
const float TEMP_AT_0V = 120.0;
const float TEMP_AT_5V = -40.0;

const float TEMP_MAX = 60.0;

byte selectArray[16][4]; // Hold 0000 in selectArray[0][0-3], 0001 in [1]
[0-3], etc.
const int selectLines[4] = { 5, 4, 3, 2 }; // Pins to multiplexer select
lines

void setup() {
    pinMode(LED_PIN, OUTPUT);
    for (int i = 0; i < 4; i++) {
        pinMode(selectLines[i], OUTPUT);
    }

    pinMode(TEMP_PIN, INPUT);

    for (int num = 0; num < 16; num++) {
        for (int bit = 3; bit >= 0; bit--) {
            selectArray[num][3 - bit] = (num >> bit) & 1; // Bit shift num
by bit times and mask the lowest bit
        }
    }

    Wire.begin();
    Serial.begin(9600);
}

void loop() {
    float highestTemp = 0;
    int highestTempChannel = 0;

    // For each MUX channel
    for (int i = 0; i < NUM_TEMP_SENSORS; i++) {
        selectMuxChannel(i);
        delayMicroseconds(20);
        float tmpTemp = analogRead(TEMP_PIN);
        // Linear interpolation will need to take place!!!!
        if (tmpTemp > highestTemp) {
            highestTemp = tmpTemp;
            highestTempChannel = i;
```

```
            }
            // Turn on LED if the temperature is too high
            if (tmpTemp > TEMP_MAX) {
                digitalWrite(LED_PIN, HIGH);
            }
        }

        // Report highest temperature via I2C
        Wire.beginTransmission(0x67);
        Wire.write(highestTemp); // Do we also want the channel number?
        Wire.endTransmission();
        delay(500);
    }

    void selectMuxChannel(int channelNum) {
        for (int i = 0; i < 4; i++) {
            digitalWrite(selectLines[i], selectArray[channelNum][i]);
        }
    }
```

6. Conclusion